Granite

A Self-Healing Multi-Agent System for Semantic Error Diagnosis, Verified Repair and Recovery

Final Year Project — Supervisor Briefing

Project Title	A Self-Healing Multi-Agent System for Semantic Error Diagnosis, Verified Repair and Recovery
Short Title	Granite
Stream	A — research-derived (starts from ErrorProbe, ACL 2026 Findings)
Prepared	2026-08-24
CCP rating	8 of 9 Seoul Accord characteristics clear
POC status	8 technical spikes run and independently reproduced
Scope	20 features: 10 foundational + 8 extensions + 2 deprioritized (§5)

1. Problem Statement

Multi-agent LLM systems are moving into production (customer-support agents, coding assistants, research tools), but the field's strongest published failure-attribution method — ErrorProbe (ACL 2026) — has two admitted, unresolved gaps: it has never been tested on a multi-agent architecture absent from its own benchmark construction, and it explicitly fails on **silent failures** — outputs that look valid but are semantically wrong, with no detectable anomaly signal. A parallel check of 7 commercial observability platforms (LangSmith, Langfuse, Braintrust, Arize, Maxim AI, Opik, DeepEval) confirmed neither gap is closed commercially either: automated diagnosis is emerging (LangSmith Engine, Arize Alyx, both 2026) but locked to each vendor's own tracing stack, with no counterfactual fix-testing and no general silent-failure detection.

Granite builds a failure-attribution and recovery system that (1) generalizes to multi-agent architectures no prior paper has tested (CrewAI, LangGraph), (2) catches silent failures via task-constraint verification, and (3) tests whether a proposed fix actually works — via counterfactual replay with a measured recovery rate — before it is applied. It ships as an installable, framework-agnostic SDK, not just a benchmark result.

2. What Is a Complex Computing Problem (CCP)?

The Seoul Accord defines a CCP as a problem with some or all of 9 characteristics (conflicting requirements, no obvious solution, depth of knowledge, unfamiliar issues, beyond standard practice, diverse stakeholders, significant consequences, interdependence, ill-defined requirements/cause). A credible CCP claim is normally supported by several of these, not all nine.

Granite clears 8 of 9:

#	Characteristic	Status	Evidence
1	Conflicting requirements	Clear	Diagnostic depth vs. latency; trace completeness vs. privacy
2	No obvious solution	Clear	Best published step accuracy still only 59.5% after 15 months, 7+ papers
3	Depth of knowledge	Clear	Execution-graph modeling, causal/counterfactual reasoning, distributed tracing
4	Unfamiliar issues	Clear	Delayed error manifestation, LLM non-determinism, cross-architecture generalization
5	Beyond standard practice	Clear	No off-the-shelf framework solves attribution + silent-failure + recovery testing
6	Diverse stakeholders	Weak — open	Present in principle; real stakeholder contact still pending
7	Significant consequences	Clear	A missed silent failure ships an incorrect answer undetected in production
8	Interdependence	Clear	Failures propagate through interdependent agent sub-problems
9	Ill-defined cause	Clear	Root cause is unknown at diagnosis time and must be investigated

3. Research Questions

#	Role	Question
RQ1	Enabling	Can our diagnosis engine's core attribution algorithm — built on ErrorProbe's published three-stage methodology — reach a defensible range of its own reported 59.5% step accuracy?
RQ2	Primary	Does our diagnosis engine generalize to multi-agent architectures absent from any of the seven surveyed papers' benchmark construction — CrewAI, native LangGraph?
RQ3	Primary	Does constraint-based output verification catch “silent failures” that ErrorProbe's own anomaly-gated pipeline admits it cannot detect?
RQ4	Secondary	Does counterfactual replay of a diagnosed root cause reliably predict which fix would have prevented the failure, measured by a real recovery rate?

RQ2 and RQ3 are the lead claims — the two confirmed gaps ErrorProbe itself admits it does not close. RQ1 is enabling (must hold for the rest to mean anything); RQ4 is secondary (depends on RQ1–3 first).

4. Proof of Concept (POC)

The guide requires at least one demonstrated feasibility proof before proposal approval. 8 spikes were run, each targeting the riskiest assumption behind one committed feature, and each independently re-run to confirm reproducibility (not a one-off result):

Spike	Tests	Result
1. ErrorProbe reproduction	Can the baseline be reproduced?	75% agent / 12.5% step accuracy (simplified pipeline, n=8) — real signal; full pipeline still needed for a fair 59.5% comparison
2. CrewAI instrumentation	Does trace capture map cleanly?	Traced cleanly into 3 nodes / 2 correctly-ordered edges — resolved
3. Fault injection	Is injected ground truth recoverable?	Confirmed via two documented behaviors: self-correction and blind propagation
4. Silent-failure detector	Does the constraint checker work?	Zero false positives on known-wrong values; natural-failure frequency still unconfirmed (n=3)
5. Counterfactual replay	Does corrected re-execution recover the outcome?	recovery_success: true, on a deterministic toy pipeline
6. Cyclic graph handling	Does backward tracing terminate on a loop?	Terminates correctly; control case proves the safety bound is load-bearing, not decorative
7. Preventive classifier	Any real signal with simple features?	Precision/recall/F1 = 0.25 — an honest negative result; mitigation plan on record, not hidden
8. Repair Agent	Can the model generate a usable fix from evidence?	repair_success_rate 0.667 (4/6), n=6, chained into Spike 5's replay logic; both failures traced to one fixable interface-specification issue, not a reasoning gap

7 of 8 mechanisms are proven to work as designed, and the 8th (spike 8, Repair Agent) works on 4 of 6 cases with a precisely diagnosed, fixable cause for the other 2 — not a vague partial result. The one clean negative result (spike 7) is reported honestly, with a stated plan to either fix it with better data/features or drop the feature — consistent with the project's standing practice of recording real findings rather than smoothing them over.

4a. Evaluation metrics (9, plan doc §11) — checked against the market survey's own Table 4

Metric	Status
Top-1 agent accuracy (= culprit-agent accuracy)	In plan
Top-1 step accuracy (= error-step accuracy)	In plan
Propagation-path F1	In plan — own addition, not in the survey's list
Silent-failure precision/recall	In plan — corrected 2026-08-24, precision was missing, recall alone can't distinguish a real detector from one that flags everything
Diagnostic latency overhead (%)	In plan — own addition
Recovery success rate (of previously-failed runs)	In plan
Task completion rate (baseline vs. self-healing, all runs)	In plan — added 2026-08-24, distinct from recovery rate, was missing from the first pass
Repair success rate	In plan — added 2026-08-24

Metric	Status
False repair rate	In plan — added 2026-08-24, the metric that keeps "verified" honest

Deliberately still excluded: re-execution cost and repair reuse benefit — both measure Checkpointing and Verified Repair Memory, neither of which is in scope; adding the metric before the feature exists would be premature, not an oversight.

5. Scope

Granite's scope is a fixed 10-item foundational core (non-negotiable — the actual research claim; was 5 items until 2026-08-23, when "core attribution algorithm" was split into its 3 real pipeline stages instead of staying one bundled label; grew to 8 on 2026-08-24, when a market survey of 7 orchestration frameworks correctly identified that nothing in the prior 7 actually generated a candidate fix — Repair Agent — then to 10 the same day, when Recovery Controller and Verified Repair Memory, briefly extensions, were reclassified into the foundational core to match the survey's own "Core" rating for both, see §6a), 8 product extensions built only after that core works and explicitly gated behind it (grew from 7 to 8 the same day with Framework Compatibility Layer, the scoped answer to "use features of all products, not only from one"; briefly reached 10 with Recovery Controller and Verified Repair Memory before those two moved into the foundational core above, settling back at 8 — see §6b), and 2 stretch items deliberately parked, not planned. The full itemized breakdown of all 20 is in §6 Features, checked against 9+ papers, 7 observability platforms, and 7 orchestration frameworks — not repeated here to avoid the same list appearing twice.

Scope type: Moderate. The mandatory core is narrow and tightly bounded — one baseline (ErrorProbe), two frameworks (CrewAI, LangGraph), one primary benchmark (Who&When). The total feature ambition across both semesters is broader, spanning a research claim and a shippable product, which is why the 8 extensions are explicitly gated behind the core rather than built in parallel — a deliberate control against breadth-over-depth (risk-register.md R8), not scope creep.

6. Features Checked Against 9+ Papers and 7 Commercial Platforms

20 features total: 10 foundational (non-negotiable, the actual research claim — was 5 until 2026-08-23, when "core attribution algorithm" was split into its 3 real pipeline stages rather than staying one bundled label; grew to 8 on 2026-08-24 with the addition of Repair Agent, then to 10 the same day when Recovery Controller and Verified Repair Memory were reclassified from extensions to foundational to match the market survey's own "Core" rating for both, see below), 8 extensions (product-differentiating, built only after the foundational 10 work — grew from 7 to 8 the same day with Framework Compatibility Layer, briefly reached 10 with Recovery Controller and Verified Repair Memory before those two moved into the foundational core, see 6b below), and 2 explicitly deprioritized (parked, not planned, no precedent check run — see 6c below). The 18 active features were each checked against 9+ academic papers, 7 observability platforms, and 7 orchestration frameworks to confirm what is genuinely new.

6a. Foundational (10) — the research claim

Feature	Precedent found?
Fault-injection harness (CrewAI, LangGraph)	Not found in any paper or product — genuinely new combination
Cross-architecture benchmark	Not found — no paper or platform publishes one
Culprit Agent (which agent)	Present, partially — ErrorProbe (paper), LangSmith Engine / Arize Alyx (commercial) — Granite extends it to unseen architectures
Error Step Localization	Present, partially — same precedent as Culprit Agent (both are outputs of the same 3-stage pipeline); no paper or platform reports step-level accuracy above 59.5%
Verification Agent (evidence-grounded)	Present, partially — this is ErrorProbe's own tool-grounded Investigator stage, reproduced not invented; no commercial platform does evidence-grounded hypothesis verification before committing to a diagnosis
Repair Agent (generates the fix)	Not found as a native capability anywhere — a 7-framework orchestration survey (OpenAI Agents SDK, LangGraph, CrewAI, Google ADK, Microsoft Agent Framework, Amazon Bedrock AgentCore, BabyAGI) rates "automatically generate repair" as Not native / Custom / Experimental across all 7; closes the gap between diagnosis and Verified Automatic Repair, which previously assumed a candidate fix was given, not generated
Semantic Error Diagnostic	Present, partially — Maxim AI / DeepEval do a narrower version (RAG-groundedness only) — Granite is general, task-constraint-based, and adds 2 new cross-referencing strategies (§7, §8)
Verified Automatic Repair	Not found — no paper or platform tests a fix with a measured recovery rate before applying it (GitHub Copilot Autofix re-scans static code with CodeQL, a different mechanism for a different failure class — see §7 note)
Recovery Controller (new, no POC)	Present, partially — 5 of 7 frameworks have generic checkpoint/resume infrastructure (LangGraph, CrewAI, Microsoft Agent Framework: checkpoint-based; Google ADK, Amazon AgentCore: partial), but none tie resumption specifically to a verified-repair point
Verified Repair Memory (re-added, one test failed)	Not found — the market survey itself rates "remember successful repairs" as "Under-served" across all 7 frameworks. Re-added by explicit user decision after one real test already failed (0/8 commits, local 3B backbone)

6b. Extensions (8) — built after the foundational 10, in priority order

Feature	Precedent found?
Widen fault taxonomy (3 types)	Supporting infrastructure — no separate precedent needed

Feature	Precedent found?
3rd architecture	Extends the same generalization claim — not a new concept
Cyclic/dynamic graph support	Not found — PROTEA (2026) names static-DAG-only as its own future work, unbuilt
Confidence calibration (published)	Not found — no paper or platform publishes a calibration study for this task
Preventive clarification classifier	Present, partially — AgentAsk (paper) does an RL-trained version — Granite's is a lighter, supervised version
Failure Propagation Map	Present broadly — most commercial platforms do some trace visualization — least differentiated item on the list
Explainable Confidence Reasoning	Not found — no paper or platform surfaces the evidence behind a confidence score
Framework Compatibility Layer (new 2026-08-24)	Not found — no framework or platform confirms lightweight execution-trace instrumentation compatibility across all 7 surveyed orchestration frameworks at once; deliberately lighter than the foundational cross-architecture benchmark (compatibility, not full benchmarking, on the frameworks outside the committed core)

6c. Deprioritized / stretch (2) — parked, not checked for precedent

Feature	Status
Cross-Domain Transfer Testing	Parked before CCP/market filtering — no precedent check run
Chaos Testing for Agents	Parked before CCP/market filtering — no precedent check run

Net answer to “were these already present, and did we add anything new”: yes to both. 7 of the 18 active features have partial precedent in either a paper or a commercial product, but never both, and never at the same generality Granite targets — 5 of those 7 are foundational (Culprit Agent, Error Step Localization, Verification Agent, Semantic Error Diagnostic, and Recovery Controller, where 5 of 7 surveyed frameworks have generic checkpoint/resume infrastructure but none tie it to a verified-repair point) and 2 are extensions (Preventive clarification classifier, Failure Propagation Map). 11 of the 18 were not found anywhere in either the academic or commercial search — including Repair Agent and Framework Compatibility Layer, both checked directly against all 7 orchestration frameworks (not just observability platforms) and rated "not native" everywhere, and Verified Repair Memory (now foundational item #10), which the survey itself rates "under-served" across the whole market and which failed its own one real test to date. The remaining 2 (deprioritized, 6c above) were parked before this filtering pass ran, so no precedent claim is made either way for them.

6d. Considered, reversed, and resolved — every decision on record, not silently dropped

Verified Repair Memory — not new: this is exactly the "Verified Episodic Memory" mechanism already built and tested once, on 2026-08-23, inside a full pipeline prototype since deleted. Real result: 0 of 8 cases committed anything to memory, because all 24 hypothesis-verifications came back unsupported on the local 3B backbone. Held off the frozen list on 2026-08-24 pending a stronger-backbone re-test — then **re-added the same day by explicit user decision**, overriding that caution rather than waiting for the re-test. Briefly placed as an extension, then **reclassified the same day into the foundational core as item #10 (§6a)**, matching the market survey's own Table 1, which rates "remember successful repairs" as "Core" for the Proposed FYP column, same as every other already-foundational item. The evidence itself hasn't changed: this item is committed to the foundational core with its one real test result still standing as a failure, not quietly reset to "untested," and not softened just because it's now core rather than an extension. **Recovery Controller** — added the same day, resolving the previously-open "Checkpointing" item with a real committed slot instead of leaving it undecided indefinitely. Briefly placed as an extension,

then **reclassified the same day into the foundational core as item #9 (§6a)**, for the same reason: the survey rates "resume from repaired point" as "Core" too. Genuinely new work, zero POC evidence yet — that status doesn't change just because it's foundational now instead of an extension. **Single-framework scope narrowing** — a market survey's own scope-control advice recommends building against one orchestration framework only. Explicitly rejected: it would eliminate RQ2, cross-architecture generalization, one of this project's two lead research claims, already CCP-justified and partially tested (Spike 2). Resolved positively, not just rejected: extension #8, Framework Compatibility Layer (§6b) — draw from all 7 surveyed frameworks via lightweight compatibility checks, while keeping full benchmarked rigor on only the 2–3 core frameworks.

6e. Second market layer checked: 7 orchestration frameworks, not just observability platforms

Source: Agentic_AI_Market_Survey_BS_FYP.docx, reviewed 2026-08-24 (full content and a section-by-section integration audit preserved in Agentic_AI_Market_Survey_Full_Content_and_Integration.md). This checks a different layer than the 7 observability platforms in §7's Use Cases and elsewhere — these are the orchestration frameworks Granite actually instruments and generalizes across.

Capability	Rated across all 7 frameworks (OpenAI SDK, LangGraph, CrewAI, Google ADK, MS Agent Framework, AWS AgentCore, BabyAGI)
Find culprit agent	Not native — all 7
Find originating error step	Not native — all 7
Causal backward error tracing	Not native — all 7
Automatically generate repair	Not native (1) / Custom (5) / Experimental self-building (1) — zero ship it first-class
Verify repair before applying	Custom or via-evaluation across all 7 — no dedicated verification stage
Resume from repaired point	Not native (2) / Checkpoint-based (3) / Partial (2) — resume exists, but not from a verified repair specifically
Remember successful repairs	Not native (1) / Custom (5) / Experimental (1) — no framework ships this

5-layer framing from the same survey, useful for positioning: (1) runtime handling and (2) workflow recovery are both rated "mostly mature" across the market — Granite reuses this layer, doesn't rebuild it. (3) semantic diagnosis is "still under-served" — Culprit Agent, Error Step Localization, Verification Agent, Semantic Error Diagnostic all sit here. (4) targeted repair is a "major gap" — Repair Agent sits here, proven 4/6 (Spike 8, risk-register.md R17). (5) verified self-healing is the survey's own named "strong FYP opportunity" — Verified Automatic Repair sits here, proven (Spike 5).

Same finding, second independent source: zero of 7 orchestration frameworks natively localize a culprit agent or error step, and none ship first-class automatic repair generation — confirming the gap Repair Agent closes, not assuming it.

7. Use Cases — What Existing Tools Miss, and What Granite Catches

Five concrete scenarios, each tied to a specific feature or mechanism above, naming the exact gap in the 7 checked commercial platforms it closes. Two of the five (#2, #3) use the silent-failure module's 2 newly-proposed cross-referencing strategies — Cross-Agent Consistency Verification and Constraint Provenance Tracing — logged 2026-08-23 in Silent_Failure_Detection_and_Testing_Strategies.md, both rule/graph-based (0% API), both still design proposals awaiting their own POC spike, not yet part of the validated core.

#	Scenario	What existing tools do	What Granite does
1	A support agent states a refund of $58.00 when the correct value, computable from the trace's own upstream data, is $61.75 — a plausible near-miss, not a crash.	LangSmith, Langfuse, Braintrust, Arize all show a clean trace — no error, no anomaly, no low-confidence flag — because nothing about the output looks wrong.	Semantic Error Diagnostic (rule-based task-constraint verification, Spike 4) recomputes the expected value from the trace's own data and flags the mismatch, without needing the true answer known in advance.
2	One agent states “12 units in stock,” a downstream agent later states “8 units in stock” for the same SKU in the same trace — each individually well-formed.	None of the 7 platforms checked cross-reference facts stated by independent agents against each other — each output is scored/traced in isolation.	Cross-Agent Consistency Verification extracts typed facts per node and diffs them across agents referencing the same entity — rule-based, no model call.
3	A final answer states “this recommendation is within your $500 budget,” but no upstream tool call in the trace ever queried a price or compared it to that budget.	Schema/type checks pass (the output is well-formed text), so nothing fires — no platform traces a claimed constraint back to a verifying node.	Constraint Provenance Tracing walks the execution graph backward from any claimed constraint and flags claims with no verifying node behind them.
4	A team builds on CrewAI, or a LangGraph workflow with native retry loops — frameworks LangSmith Engine and Arize Alyx were not tuned for.	LangSmith Engine and Arize “Alyx” (2026) both ship real automated root-cause diagnosis, but locked to their own vendor tracing stack.	Framework-agnostic by design — proven on CrewAI (Spike 2) and on LangGraph's native retry loops (Spike 6, load-bearing cycle-safety bound).
5	A diagnosis is confirmed (Verification Agent), but nobody has proposed what the actual fix should be, let alone tested whether it would work.	Across all 7 orchestration frameworks surveyed, "automatically generate repair" rates Not native / Custom / Experimental — none ship a first-class repair-generation step.	Repair Agent generates the targeted correction (prompt, tool argument, structured output, or function logic); Verified Automatic Repair (Spike 5, recovery_success: true) then re-executes downstream with it and reports a measured recovery rate before it's trusted.

8. Each Feature, the CCP Characteristic It Evidences, and Who Benefits

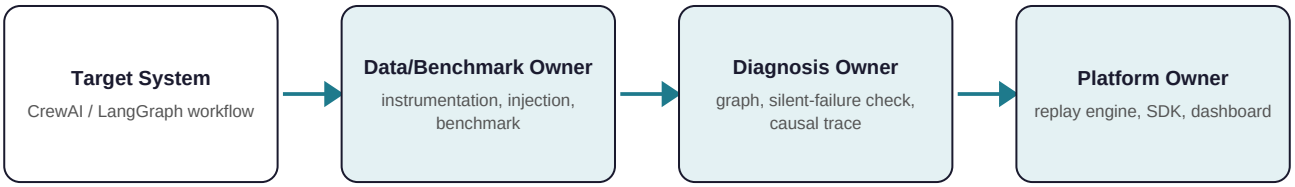
Every one of the 18 frozen features maps to at least one of the 9 Seoul Accord CCP characteristics (§2) and a concrete beneficiary — not an abstract “users,” a named role who would actually reach for this specific capability.

Feature	CCP characteristic(s)	Who benefits
Fault-injection harness	#3 Depth of knowledge, #5 Beyond standard practice	Benchmark researchers; QA/test engineers validating agent systems pre-deployment
Cross-architecture benchmark	#2 No obvious solution, #6 Diverse stakeholders	Researchers comparing attribution methods; teams choosing which framework to trust
Culprit Agent (which agent)	#4 Unfamiliar issues, #9 Ill-defined cause	On-call engineers / SREs diagnosing a production multi-agent failure
Error Step Localization (which step)	#4 Unfamiliar issues, #9 Ill-defined cause	On-call engineers / SREs needing the exact origin point, not just the responsible agent
Verification Agent (evidence-grounded)	#3 Depth of knowledge, #9 Ill-defined cause	Engineers who need a diagnosis backed by trajectory evidence, not just a model's assertion
Repair Agent (generates the fix)	#2 No obvious solution, #3 Depth of knowledge	Engineers who want a proposed correction, not just a diagnosis, before deciding what to do
Semantic Error Diagnostic	#7 Significant consequences, #9 Ill-defined cause	End users receiving agent outputs (support, finance); compliance/QA teams
Verified Automatic Repair	#1 Conflicting requirements, #8 Interdependence	Engineers deciding whether to trust and apply a suggested fix; incident post-mortems
Recovery Controller	#5 Beyond standard practice, #8 Interdependence	Engineers who need execution resumed from a verified-repair checkpoint instead of a full restart; ops teams cutting recovery time
Verified Repair Memory	#3 Depth of knowledge, #5 Beyond standard practice	Engineers hitting repeated/similar failures who want faster diagnosis-and-repair reuse; teams building institutional repair knowledge
Widen fault taxonomy	#3 Depth of knowledge	Benchmark users needing broader failure-type coverage
3rd architecture	#4 Unfamiliar issues, #6 Diverse stakeholders	Teams on OpenAI Agents SDK or a custom in-house framework
Cyclic/dynamic graph support	#8 Interdependence	Teams using LangGraph's native retry / refine loops
Confidence calibration	#1 Conflicting requirements	Teams deciding how much to trust automated diagnosis before acting on it
Preventive clarification classifier	#7 Significant consequences	Engineers wanting to catch corruption before it propagates, not just after
Failure Propagation Map	#8 Interdependence	Engineers visualizing how a failure spread across interdependent agents
Explainable Confidence Reasoning	#5 Beyond standard practice	Engineers/auditors who must justify why a diagnosis was trusted
Framework Compatibility Layer	#1 Conflicting requirements, #6 Diverse stakeholders	Teams on any of the 7 surveyed frameworks who want confirmed instrumentation compatibility without waiting for full benchmarked support

Feature	CCP characteristic(s)	Who benefits
Cross-Domain Transfer Testing (deprioritized)	#4 Unfamiliar issues	Teams deploying the same diagnosis engine across multiple task domains
Chaos Testing for Agents (deprioritized)	#7 Significant consequences	SRE/ops teams stress-testing production resilience

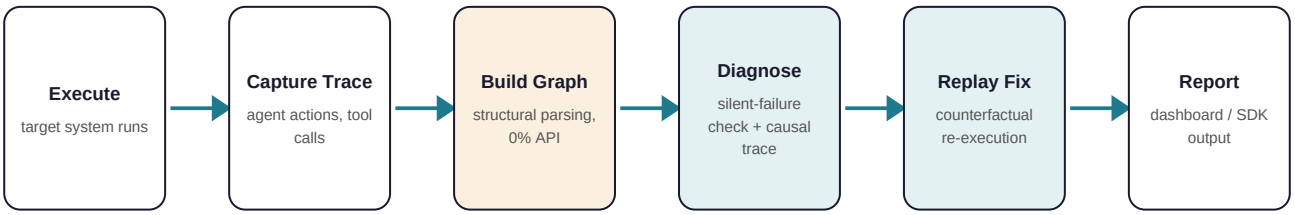
9. Architecture & Workflow

Ownership architecture — who builds what:



Three role areas map directly to the three subsystems above — not yet assigned to specific team members (§11 Team Ownership). Every component inside Diagnosis Owner's box is committed to rule-based or graph-based own logic, not an LLM call.

Data-flow workflow — one execution, step by step:



This is the sequence every diagnosed failure passes through, proven stage-by-stage in the 8 POC spikes: Capture (spike 2), Build Graph (spike 2, spike 6 for cycles), Diagnose (spikes 1 and 4), Generate Fix (spike 8), Replay Fix (spike 5). Report is the one stage with no dedicated spike, since it is pure display over already-validated data.

10. Feasibility, Relevant to the FYP Guide

Checked against the FAST FYP Selection Guide's own criteria: AI-resilience test (§7), external-component classification (§14), and weak-idea patterns (§15).

AI-resilience test:

“Could a strong student with AI assistance build a working version of this in a week?” — No. The remaining difficulty sits in data engineering (standing up real multi-agent frameworks, fault injection at scale), evaluation design (a fair, reproducible ErrorProbe comparison), and deployment reality — exactly where the guide says a defensible FYP's difficulty must live.

External components:

The LLM API is classified as Core-assist, not Core-replacement, with a committed ~15–20% API-dependency budget. Every core mechanism — fault injection, graph construction, causal tracing, confidence scoring, and critically the silent-failure verification module itself — is rule-based or graph-based own logic, not an LLM call. If the LLM were swapped, these components would be entirely unaffected.

Resourcing:

Achievable at $0–near-$0: open-weight self-hosted models (GPT-OSS-120B, Qwen3-32B — the same backbones ErrorProbe itself used), free-tier cloud inference (Google AI Studio, Groq), and fully public, license-verified datasets (Who&When;, MBPP+, GSM8K, tau-bench retail). All 8 POC spikes above were run entirely on local, free compute.

11. Team Ownership

Three defined role areas, not assignments — no specific team member has been mapped to a role yet; that decision is real and still open, not a formality.

Role	Owns	If removed
Data/Benchmark Owner (not yet assigned)	Multi-framework instrumentation, injection operators, ground-truth benchmark, licensing & release	No cross-architecture evaluation is possible
Diagnosis Owner (not yet assigned)	Execution-graph construction, the ErrorProbe-based attribution algorithm, silent-failure verification, causal tracing	The core attribution and silent-failure claims disappear
Platform Owner (not yet assigned)	Counterfactual replay, SDK, backend, containerized deployment, dashboard	Nothing is installable outside the dev machine

12. What Remains Open

- Real stakeholder contact — the one item most directly tied to CCP characteristic #6
- Assigning the 3 defined ownership roles (Data/Benchmark, Diagnosis, Platform) to actual team members
- Checking the department's own prior-FYP archive for structural overlap
- Each team member independently re-running their owned spikes and explaining them without notes

- POC-testing the 2 newly-proposed silent-failure strategies (Cross-Agent Consistency Verification, Constraint Provenance Tracing) before they can be added to the validated core
- Repair Agent's 2 failing cases (risk-register.md R17) — both traced to one fixable interface-specification issue (the model returns a computed result instead of the specific corrected parameter); worth a targeted prompt fix and re-test before Mid-1, not a re-run from scratch
- A re-test of Verified Repair Memory (risk-register.md R18) on a stronger backbone before any decision to re-add it

13. Appendix — Full Market Survey Reference

Full verbatim content of *Agentic_AI_Market_Survey_BS_FYP.docx* (reviewed 2026-08-24), reproduced here so this briefing is self-contained and does not require jumping to markdowns/03-market-and-literature-research/Agentic_AI_Market_Survey_Full_Content_and_Integration.md for the underlying detail behind every claim cited throughout §6–§7 above. All 12 source headings, all 4 tables, and all 8 platform write-ups follow, verbatim, not paraphrased.

Executive Summary

Purpose. This survey compares major agentic AI frameworks and platforms against the capabilities required for a proposed self-healing multi-agent system. The analysis distinguishes ordinary operational error handling (exceptions, retries, checkpoints and tracing) from semantic root-cause diagnosis (which agent introduced the error and at which earlier step), and from closed-loop recovery (repair, verification, resumption and learning).

Key finding. The market is mature for orchestration, tool calling, traces, retries, guardrails, memory and checkpoint-based recovery. However, the surveyed products do not natively provide the complete closed loop of: semantic failure detection -> culprit-agent attribution -> originating-step localisation -> targeted repair -> verification -> resume from a corrected state -> retain the successful repair for future use.

Recommended FYP position. Use an existing framework such as LangGraph, Google ADK, CrewAI or Microsoft Agent Framework for commodity orchestration and persistence. Concentrate the student contribution on four modules: Root-Cause Localizer, Repair Agent, Verification Agent and Verified Repair Memory.

Core research gap: Existing SDKs can observe, retry and resume workflows; ERRORPROBE can attribute semantic failures; the missing integration is to turn a verified diagnosis into a verified repair and resume execution from the corrected state.

1. Scope and Evaluation Criteria

The survey evaluates frameworks at three layers: **Operational robustness** — runtime errors, tool failures, retries, guardrails, tracing and checkpoints. **Semantic diagnosis** — identifying that an apparently valid result is wrong, locating the responsible agent and finding the originating interaction step. **Self-healing recovery** — generating a targeted correction, verifying it, resuming execution and reusing verified repairs in future failures.

Interpretation of symbols. Built-in/strong support = native or first-class capability; Partial/custom = feasible through developer logic or ecosystem extensions; Experimental = available but not mature; Not native = not provided as a first-class capability by the framework.

2. Comparative Feature Matrix

Table 1. Market comparison of agentic AI platforms against the proposed self-healing FYP.

Feature	OpenAI Agents SDK	LangGraph	CrewAI	Google ADK	MS Agent Framework	Amazon AgentCore	BabyAGI	Proposed FYP
Multi-agent orchestration	Strong	Strong	Strong	Strong	Strong	Platform/runtime	Partial	Strong
Tool calling	Strong	Strong	Strong	Strong	Strong	Strong	Strong	Strong
Execution tracing	Strong	Strong ecosystem	Strong	Strong	Strong	Strong	Strong logs	Strong
Runtime-error detection	Strong	Strong	Strong	Strong	Strong	Strong	Strong logs	Strong
Automatic retry	Partial/custom	Strong	Partial/custom	Strong	Strong/custom	Operational retries	Partial	Strong
Guardrails / validation	Strong	Partial/custom	Strong	Strong	Strong	Strong	Limited	Strong
Persistent memory	Strong	Strong	Strong	Strong	Strong	Strong	Strong	Strong
Checkpoint / resume	Partial	Strong	Strong	Workflow support	Strong	Runtime-oriented	Limited	Strong
Workflow replay / fork	Partial	Strong	Strong	Evaluation/replay	Strong	Partial	Not native	Strong
Agent evaluation	Strong	Ecosystem	Strong	Strong	Strong	Strong	Limited	Strong
Detect semantic wrong answer	Partial via eval/guardrail	Partial/custom	Partial/custom	Evaluation support	Partial/custom	Evaluation support	Limited	Core
Find culprit agent	Not native	Not native	Not native	Not native	Not native	Not native	Not native	Core
Find originating error step	Not native	Not native	Not native	Not native	Not native	Not native	Not native	Core
Causal backward error tracing	Not native	Not native	Not native	Not native	Not native	Not native	Not native	Core
Automatically generate repair	Not native	Custom	Custom	Custom	Custom	Custom	Experimental self-building	Core
Verify repair before applying	Custom	Custom	Custom	Via evaluation/custom	Custom	Via evaluation/custom	Limited	Core
Resume from repaired point	Not native	Checkpoint-based	Checkpoint-based	Partial	Checkpoint-based	Partial	Not native	Core
Remember successful repairs	Not native	Custom	Custom	Custom	Custom	Custom	Experimental	Core
Self-improve from prior failures	Not native	Partial/custom	Partial/custom	Partial/custom	Partial/custom	Partial/custom	Experimental	Verified/controlled

Reading the table correctly: A "Not native" rating does not mean a capability is impossible to implement; it means the framework does not provide it as a first-class, built-in feature. This distinction is important because the proposed project should not claim novelty for functionality that can simply be assembled through standard framework hooks.

3. Platform and Framework Survey

3.1 OpenAI Agents SDK

Provides multi-agent handoffs, tool integration, sessions, guardrails, tracing and evaluation-oriented instrumentation. Its tracing model can capture agent turns, model generations, tool calls, handoffs and guardrails. Tool guardrails can validate calls before and after execution. For this FYP, these features should be treated as infrastructure rather than novelty. The SDK does not natively perform causal attribution to identify the earlier agent/step that introduced a semantic failure, nor does it automatically repair and replay that component.

3.2 LangGraph

Particularly suitable for resilient workflows because it provides durable state, checkpointing, fault tolerance, retry policies, persistence and time-travel/replay. This makes it a strong implementation substrate for the proposed FYP. The student can reuse LangGraph for checkpoint/resume and focus on semantic diagnosis and verified repair. LangGraph can tell the system where execution failed, but causal semantic attribution to an earlier agent remains a separate research problem.

3.3 CrewAI

Provides multi-agent orchestration, tracing/observability and checkpoint-oriented execution features. Its recovery mechanisms can save state and resume a workflow rather than repeating all completed work. It therefore covers a large portion of operational fault tolerance. However, identifying the semantic root cause of a seemingly valid but wrong multi-agent trajectory and automatically repairing the responsible step remains custom logic.

3.4 Google Agent Development Kit (ADK)

Offers agent orchestration, plugins, callbacks, evaluation and retry-related mechanisms. Its evaluation tooling is relevant because it can assess final answers and trajectories/tool-use sequences. This reduces the novelty of simply adding an "agent evaluator." The proposed FYP should instead use evaluation as a verification signal and focus on causal localization plus targeted repair.

3.5 Microsoft Agent Framework

Provides graph-based multi-agent workflows, checkpointing, state management, human-in-the-loop mechanisms, middleware and telemetry. These are strong foundations for recovery and production engineering. The remaining gap is not basic recovery from an exception, but determining the semantic origin of a delayed failure and transforming that diagnosis into an automatically verified correction.

3.6 Amazon Bedrock AgentCore

Strongest at enterprise runtime, observability and evaluation. It provides logs, traces, metrics and production monitoring for agent workloads. This is highly useful for collecting evidence about failures, but observability alone does not solve semantic causal attribution. A workflow may complete successfully at the infrastructure level while still returning a semantically wrong result.

3.7 BabyAGI

Useful as conceptual inspiration because its function-centric architecture records execution details and explores experimental self-building capabilities. In principle, self-building allows a system to create or revise capabilities rather than only report an error. However, BabyAGI does not provide the same systematic, benchmarked culprit-agent and originating-step diagnosis studied by ERRORPROBE. For an

FYP, its self-modification idea is more useful as inspiration for the Repair Agent than as a direct baseline for semantic attribution.

3.8 ERRORPROBE (ACL Findings 2026)

A research framework for semantic failure attribution in multi-agent systems. It takes a failed trace, uses structural decomposition and symptom-driven backward tracing, and employs a Strategist-Investigator-Arbiter team to identify the responsible agent and decisive error step. It also stores verified diagnostic patterns. Its main contribution is therefore diagnosis rather than closed-loop operational recovery: the framework improves the debugger, but it does not demonstrate a complete system that automatically repairs the responsible component, replays the workflow from a corrected state and retains verified repairs as executable recovery knowledge.

4. Error-Handling Layers: Why the Systems Are Complementary

Table 2. Five layers of agentic error handling and the location of the proposed project gap.

Layer	Typical Question	Strong Existing Solutions	Remaining Gap
1. Runtime handling	Did a tool/API/node fail?	Agent SDKs, LangGraph, AgentCore, ADK, CrewAI	Mostly mature
2. Workflow recovery	Can execution retry/resume?	LangGraph, CrewAI, Microsoft Agent Framework	Mostly mature
3. Semantic diagnosis	Which agent introduced the wrong reasoning and where?	ERRORPROBE; research prototypes	Still under-served
4. Targeted repair	Can the responsible component be corrected automatically?	Custom logic; experimental self-building	Major gap
5. Verified self-healing	Can the fix be tested, replayed and remembered safely?	Partial/custom across frameworks	Strong FYP opportunity

5. Market Gap Analysis

Table 3. Market saturation versus potential FYP contribution.

Capability	Market Status	FYP Value
Agent orchestration	Saturated	Reuse existing framework
Tool integration	Saturated	Reuse
Logging/tracing	Saturated	Reuse
Exception detection	Saturated	Reuse
Automatic retries	Saturated	Reuse
Memory	Saturated	Reuse
Checkpoint/resume	Strong existing solutions	Reuse
Guardrails	Strong existing solutions	Reuse
Agent evaluation	Rapidly maturing	Reuse/adapt
Semantic failure detection	Partial	Useful component
Culprit-agent attribution	Research-level / under-served	Strong feature
Originating-step localisation	Research-level / under-served	Strong feature
Automatic targeted repair	Major gap	Excellent feature
Verification of generated repair	Partial/custom	Excellent feature
Resume after verified repair	Integration gap	Excellent feature
Verified repair memory	Under-served	Excellent feature

Gap statement suitable for the proposal: Existing agent SDKs provide mechanisms for observing, retrying and resuming failed workflows, while recent research such as ERRORPROBE addresses semantic failure attribution. A practical gap remains in automatically transforming root-cause diagnosis into a verified repair, resuming execution from the corrected state, and retaining successful repair knowledge for future failures.

6. Recommended BS Final Year Project

Proposed Title: *A Self-Healing Multi-Agent System for Semantic Error Diagnosis, Verified Repair and Recovery*

6.1 Recommended System Flow

(Figure 1: Execute MAS -> Detect Failure -> Localise Cause -> Generate Repair -> Verify Repair -> Resume Workflow -> Store Repair)

1. **Multi-Agent Runtime:** Use an existing framework for Planner/Worker/Reviewer or similar agents.
2. **Trace Collector:** Store agent messages, tool calls, outputs, exceptions and checkpoint identifiers.
3. **Semantic Failure Detector:** Identify failures that may not be ordinary runtime exceptions, using constraints, evaluator signals or task-specific checks.
4. **Root-Cause Localizer:** Estimate the culprit agent and originating error step using backward dependency/trace reasoning.
5. **Repair Agent:** Generate a targeted correction to the prompt, tool arguments, structured output or function logic associated with the localized failure.
6. **Verification Agent:** Test candidate repairs using deterministic checks, sandbox execution, test cases or evaluator constraints.
7. **Recovery Controller:** Apply only verified repairs and resume execution from a safe checkpoint instead of restarting the entire workflow.
8. **Verified Repair Memory:** Store successful error -> diagnosis -> repair patterns and retrieve them for similar future failures.
9. **Dashboard:** Visualize the workflow, failed agent, originating step, repair attempt, verification result and final recovered execution.

7. Scope Control for a One-Year BS Project

The complete research vision can become too broad. For a realistic one-year implementation, the project should deliberately constrain the environment:

- Use one agent framework only (recommended: LangGraph, because checkpointing and replay are already available).
- Use one or two task domains, such as code/tool-use tasks and structured reasoning tasks.
- Limit the MAS to approximately 3–4 roles so causal traces remain understandable.
- Implement a small taxonomy of failure types rather than attempting every possible semantic failure.
- Treat repair as targeted modification of prompts, tool arguments, structured outputs or small code functions; do not attempt unrestricted rewriting of the entire agent architecture.
- Require verification before any generated repair is applied or added to memory.

Note on the first bullet — "use one framework only": this specific recommendation was explicitly considered and rejected by Granite (risk-register.md R19), since it would eliminate RQ2 (cross-architecture generalization), one of this project's two lead research questions. It is not silently ignored: it is replaced by extension #8, **Framework Compatibility Layer** — full fault-injection and benchmarked rigor stays on only the 2–3 committed core frameworks (CrewAI, LangGraph, +1), while lightweight compatibility checks (can the pipeline instrument and parse execution traces at all) extend across all 7 surveyed frameworks. Broad *compatibility*, not broad *benchmarking* — the honest, scoped answer to "use features of all products, not only from one" without doubling the core's engineering workload. See §6b above and risk-register.md R20.

8. Evaluation Plan

Table 4. Recommended quantitative evaluation metrics for the FYP.

Metric	What It Measures	Suggested Comparison
Task completion rate	Whether the full task succeeds	Baseline MAS vs. self-healing MAS
Failure detection precision/recall	Quality of semantic failure detection	Detector variants
Culprit-agent accuracy	Correct responsible agent	Root-cause module vs. simple last-error baseline
Error-step accuracy	Correct originating step	Backward tracing vs. full-trace judge
Repair success rate	Percentage of generated repairs that fix the task	With vs. without verification
Recovery rate	Failed runs converted into successful runs	Baseline retry vs. targeted repair
Re-execution cost	Tokens/tool calls/time after failure	Restart-from-zero vs. checkpoint resume
Repair reuse benefit	Improvement when verified memory is enabled	Memory off vs. memory on
False repair rate	Incorrect repairs accepted by system	Verification ablation

9. What the Students Should and Should Not Claim

Strong, defensible contribution:

- Integration of semantic root-cause localization with automatic targeted repair.
- Verification-gated repair acceptance rather than unconstrained LLM self-modification.
- Checkpoint-based recovery from the corrected state.
- A verified repair memory evaluated for reuse across repeated/similar failure patterns.

Claims to avoid:

- "First agent system with retries, tracing or memory." These are already common capabilities.
- "BabyAGI/AgentCore/LangGraph cannot handle errors." They handle many operational errors well; the gap is semantic causal diagnosis and closed-loop repair.
- "The system solves all hallucinations." Silent semantic failures remain a difficult open problem.
- "The FYP outperforms ERRORPROBE" unless the students reproduce a fair benchmark comparison.

10. Conclusion

The agentic AI market already provides mature building blocks for orchestration, tools, observability, retries, guardrails, persistent state and checkpoint-based recovery. A BS project should therefore avoid rebuilding those features. The strongest remaining FYP opportunity is the integration between semantic diagnosis and recovery: identify the real originating agent/step, generate a targeted repair, verify that repair, resume from a safe checkpoint and remember only successful repairs. This produces a technically meaningful "self-healing" contribution while keeping the project achievable within one academic year.

References and Official Technical Sources

- Li, J., Yilmaz, E., Chen, B. and Le, D.-T. (2026). "Towards Self-Improving Error Diagnosis in Multi-Agent Systems." Findings of the Association for Computational Linguistics: ACL 2026, pp. 2063–2077. https://aclanthology.org/2026.findings-acl.98/
- OpenAI. Agents SDK documentation: Agents, tracing and guardrails. https://openai.github.io/openai-agents-python/
- LangChain. LangGraph documentation: Persistence, checkpointing and fault tolerance. https://docs.langchain.com/oss/python/langgraph/persistence
- LangChain. LangGraph RetryPolicy reference. https://reference.langchain.com/python/langgraph/types/RetryPolicy
- CrewAI. Documentation: Checkpointing and Observability. https://docs.crewai.com/en/concepts/checkpointing
- Google. Agent Development Kit documentation: Plugins and evaluation. https://google.github.io/adk-docs/
- Microsoft. Agent Framework documentation: Workflows and checkpoints. https://learn.microsoft.com/en-us/agent-framework/workflows/
- Amazon Web Services. Amazon Bedrock AgentCore documentation: Observability and evaluations. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/
- Nakajima, Y. BabyAGI repository and documentation. https://github.com/yoheinakajima/babyagi

Source note: The ERRORPROBE analysis in this report is grounded in the ACL 2026 paper supplied for the discussion. Product/framework capabilities are summarized from their official documentation and repositories. Feature ratings represent whether a capability is native/first-class versus requiring custom implementation, not whether it is theoretically possible.